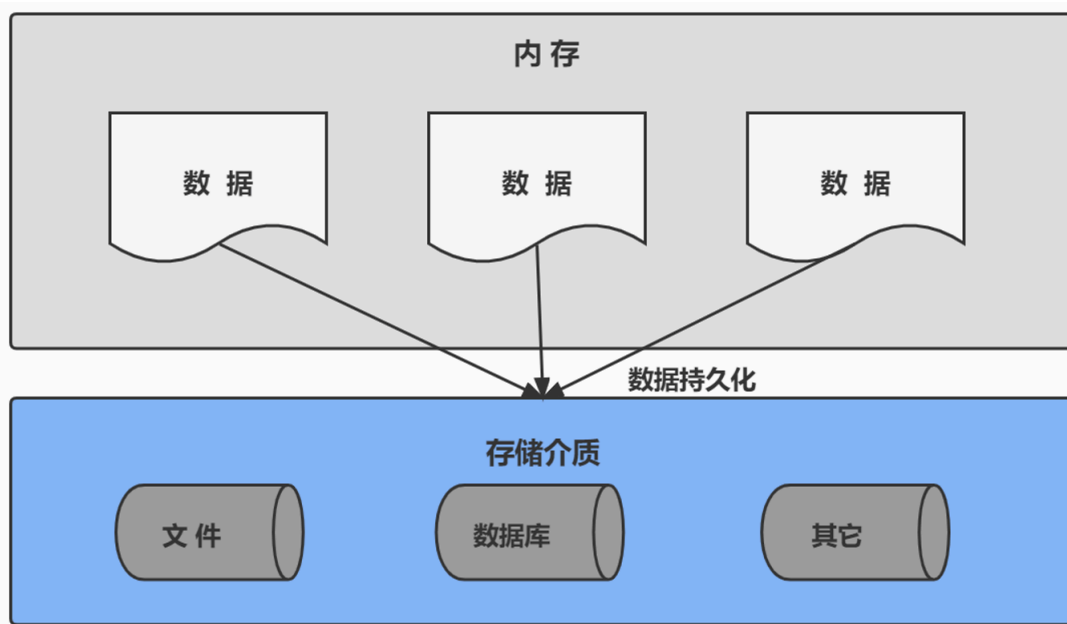


JDBC核心技术

第1章：JDBC概述

1.1 数据的持久化

- 持久化(persistence)：把数据保存到可掉电式存储设备中以便之后使用。大多数情况下，特别是企业级应用，数据持久化意味着将内存中的数据保存到硬盘上加以“固化”，而持久化的实现过程大多通过各种关系数据库来完成。
- 持久化的主要应用是将内存中的数据存储在关系型数据库中，当然也可以存储在磁盘文件、XML数据文件中。



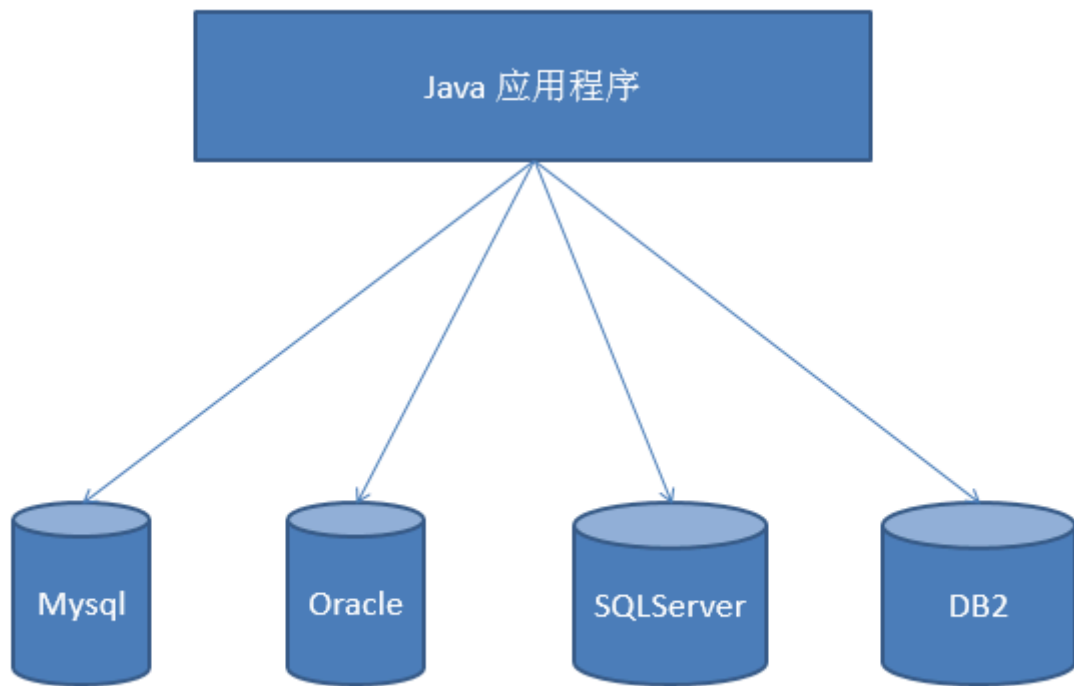
1.2 Java中的数据存储技术

- 在Java中，数据库存取技术可分为如下几类：
 - JDBC直接访问数据库
 - JDO (Java Data Object) 技术
 - 第三方O/R工具，如Hibernate, Mybatis 等
- JDBC是Java访问数据库的基石，[JDO](#)、Hibernate、MyBatis等只是更好的封装了JDBC。

1.3 JDBC介绍

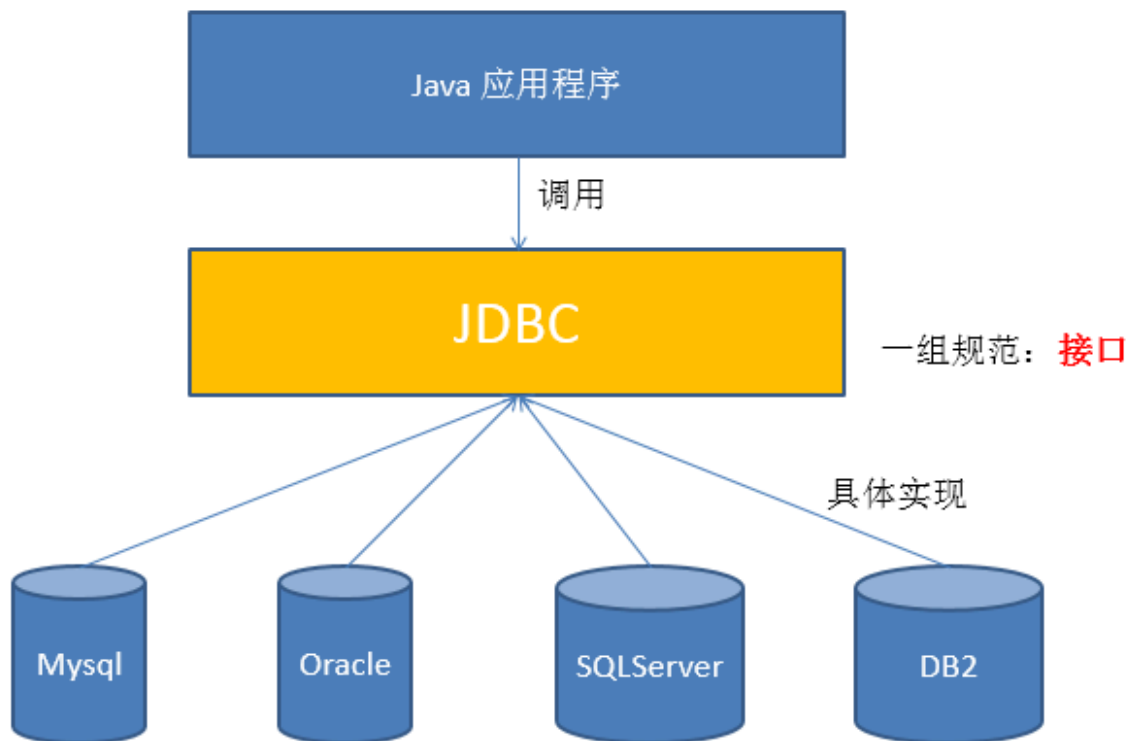
- JDBC(Java Database Connectivity)是一个独立于特定数据库管理系统、通用的SQL数据库存取和操作的公共接口（一组API），定义了用来访问数据库的标准Java类库，（`java.sql`, `javax.sql`）使用这些类库可以以一种标准的方法、方便地访问数据库资源。
- JDBC：sun公司提供的一套API，使用这套API可以实现对数据库的管理操作。（获取连接、CRUD等）
- JDBC为访问不同的数据库提供了一种统一的途径，为开发者屏蔽了一些细节问题。
- JDBC的目标是使Java程序员使用JDBC可以连接任何提供了JDBC驱动程序的数据库系统，这样就使得程序员无需对特定的数据库系统的特点有过多的了解，从而大大简化和加快了开发过程。

- 如果没有JDBC, 那么Java程序访问数据库时是这样的:



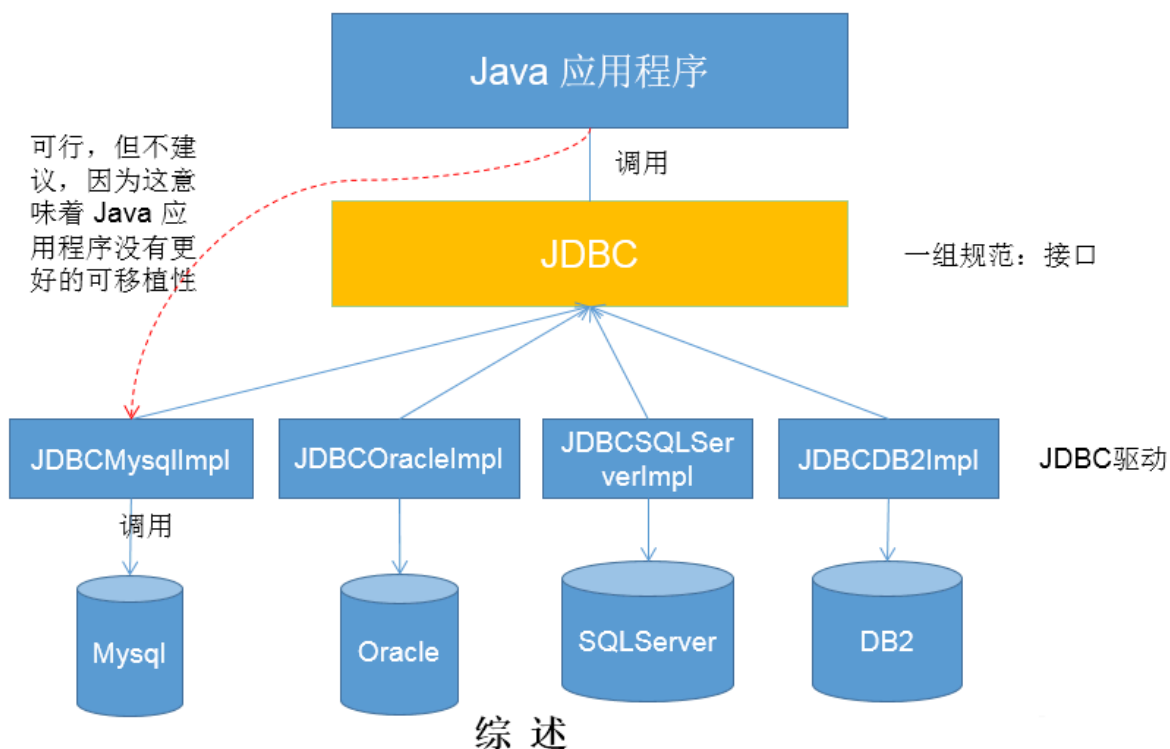
我们认为的连接应该这样

- 有了JDBC, Java程序访问数据库时是这样的:



真实的连接是这样

- 总结如下:



1.4 JDBC体系结构

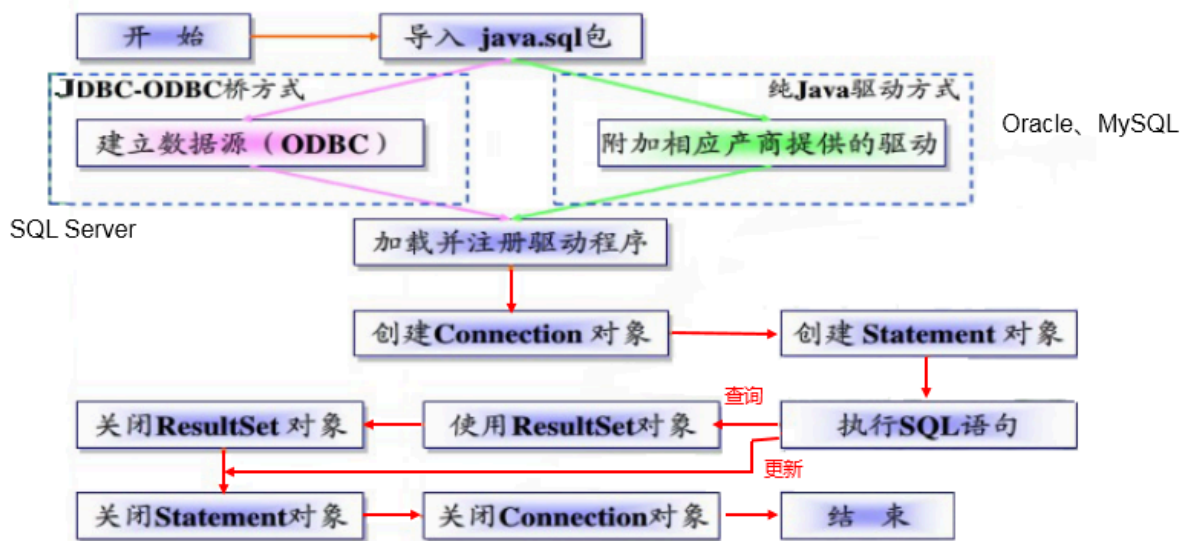
- JDBC接口 (API) 包括两个层次：
 - 面向应用的API**：Java API，抽象接口，供应用程序开发人员使用（连接数据库，执行SQL语句，获得结果）。
 - 面向数据库的API**：Java Driver API，供开发商开发数据库驱动程序用。

JDBC是sun公司提供一套用于数据库操作的接口，java程序员只需要面向这套接口编程即可。

不同的数据库厂商，需要针对这套接口，提供不同实现。不同的实现的集合，即为不同数据库的驱动。

———面向接口编程

1.5 JDBC程序编写步骤



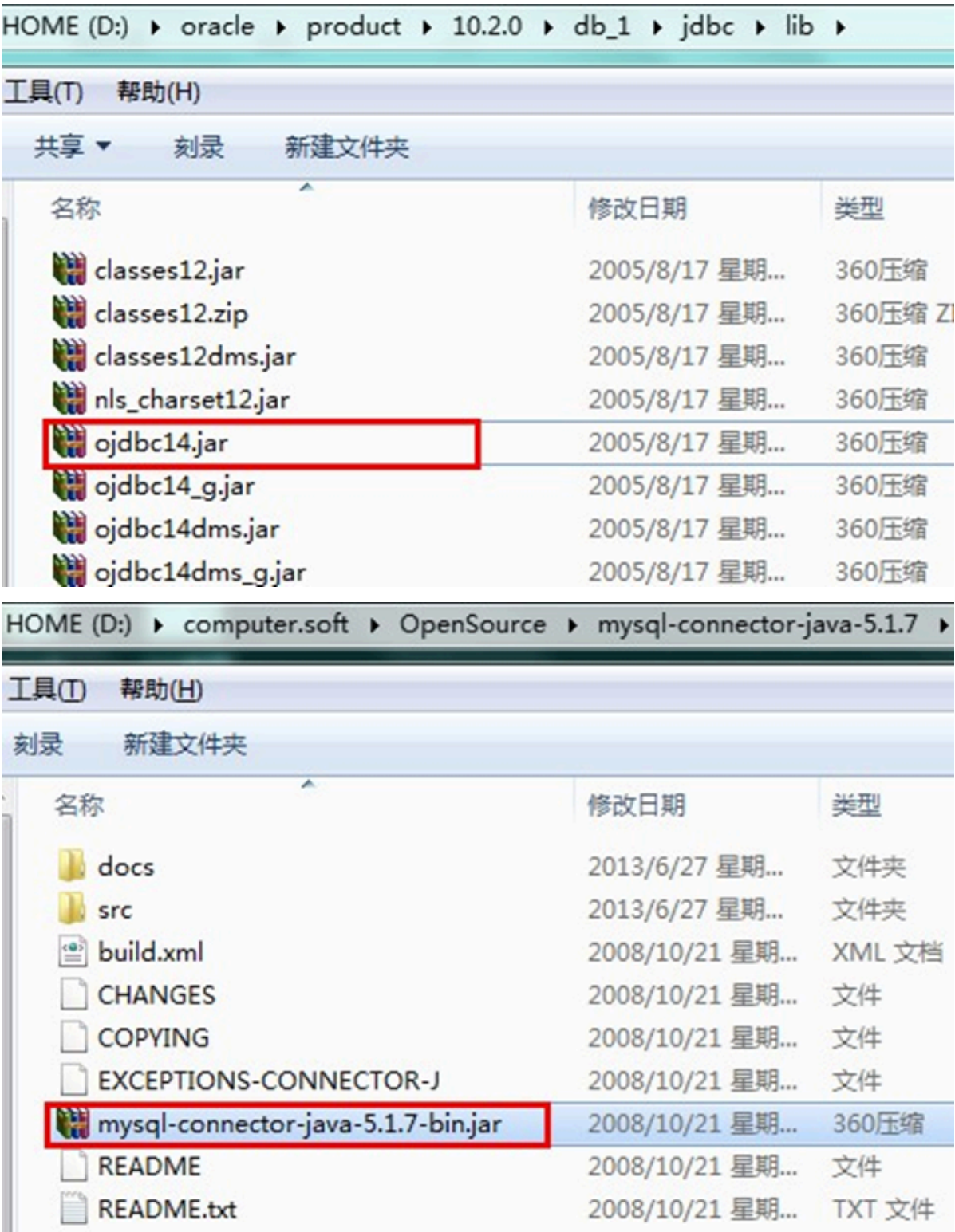
补充：ODBC(**O**pen **D**atabase **C**onnectivity，开放式数据库连接)，是微软在Windows平台下推出的。使用者在程序中只需要调用ODBC API，由 ODBC 驱动程序将调用转换成为对特定的数据库的调用请求。

第2章：获取数据库连接

2.1 要素一：Driver接口实现类

2.1.1 Driver接口介绍

- java.sql.Driver 接口是所有 JDBC 驱动程序需要实现的接口。这个接口是提供给数据库厂商使用的，不同数据库厂商提供不同的实现。
- 在程序中不需要直接去访问实现了 Driver 接口的类，而是由驱动程序管理器类 (java.sql.DriverManager)去调用这些Driver实现。
 - Oracle的驱动：`oracle.jdbc.driver.OracleDriver`
 - mySql的驱动：`com.mysql.jdbc.Driver`



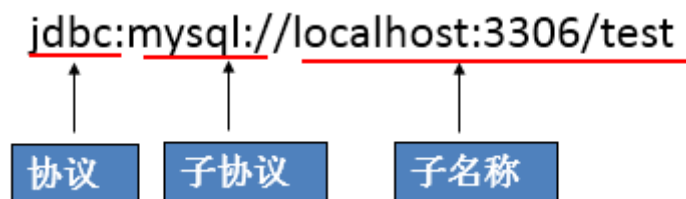
2.1.2 加载与注册JDBC驱动

- 加载驱动：加载 JDBC 驱动需调用 Class 类的静态方法 `forName()`，向其传递要加载的 JDBC 驱动的类型名
 - `Class.forName("com.mysql.cj.jdbc.Driver");`
- 注册驱动：DriverManager 类是驱动程序管理器类，负责管理驱动程序
 - 使用 `DriverManager.registerDriver(com.mysql.cj.jdbc.Driver)` 来注册驱动
 - 通常不用显式调用 DriverManager 类的 `registerDriver()` 方法来注册驱动程序类的实例，因为 Driver 接口的驱动程序类都包含了静态代码块，在这个静态代码块中，会调用 `DriverManager.registerDriver()` 方法来注册自身的一个实例。下图是MySQL的Driver实现类的源码：

```
public class Driver extends NonRegisteringDriver implements java.sql.Driver {  
    //  
    // Register ourselves with the DriverManager  
    //  
    static {  
        try {  
            java.sql.DriverManager.registerDriver(new Driver());  
        } catch (SQLException E) {  
            throw new RuntimeException("Can't register driver!");  
        }  
    }  
}
```

2.2 要素二：URL

- JDBC URL 用于标识一个被注册的驱动程序，驱动程序管理器通过这个 URL 选择正确的驱动程序，从而建立到数据库的连接。
- JDBC URL的标准由三部分组成，各部分间用冒号分隔。
 - **jdbc:子协议:子名称**
 - **协议**：JDBC URL中的协议总是jdbc
 - **子协议**：子协议用于标识一个数据库驱动程序
 - **子名称**：一种标识数据库的方法。子名称可以依不同的子协议而变化，用子名称的目的是为了**定位数据库**提供足够的信息。包含**主机名**(对应服务端的ip地址)，**端口号**，**数据库名**
- 举例：



- 几种常用数据库的 JDBC URL
 - MySQL的连接URL编写方式：
 - `jdbc:mysql://主机名称:mysql服务端口号/数据库名称?参数=值&参数=值`
 - `jdbc:mysql://localhost:3306/atguigu`
 - `jdbc:mysql://localhost:3306/atguigu ?useUnicode=true&characterEncoding=utf8`
(如果JDBC程序与服务器端的字符集不一致，会导致乱码，那么可以通过参数指定服务器端的字符集)
 - 针对于MySQL8.0的连接，需要在输入地址后面必须加入 `useSSL=false&serverTimezone=UTC` 确认时区。即：
`jdbc:mysql://localhost:3306/test?useSSL=false&serverTimezone=UTC`

- Oracle 9i的连接URL编写方式:
 - jdbc:oracle:thin:@主机名称:oracle服务端口号:数据库名称
 - jdbc:oracle:thin:@localhost:1521:atguigu
- SQLServer的连接URL编写方式:
 - jdbc:sqlserver://主机名称:sqlserver服务端口号:DatabaseName=数据库名称
 - jdbc:sqlserver://localhost:1433:DatabaseName=atguigu

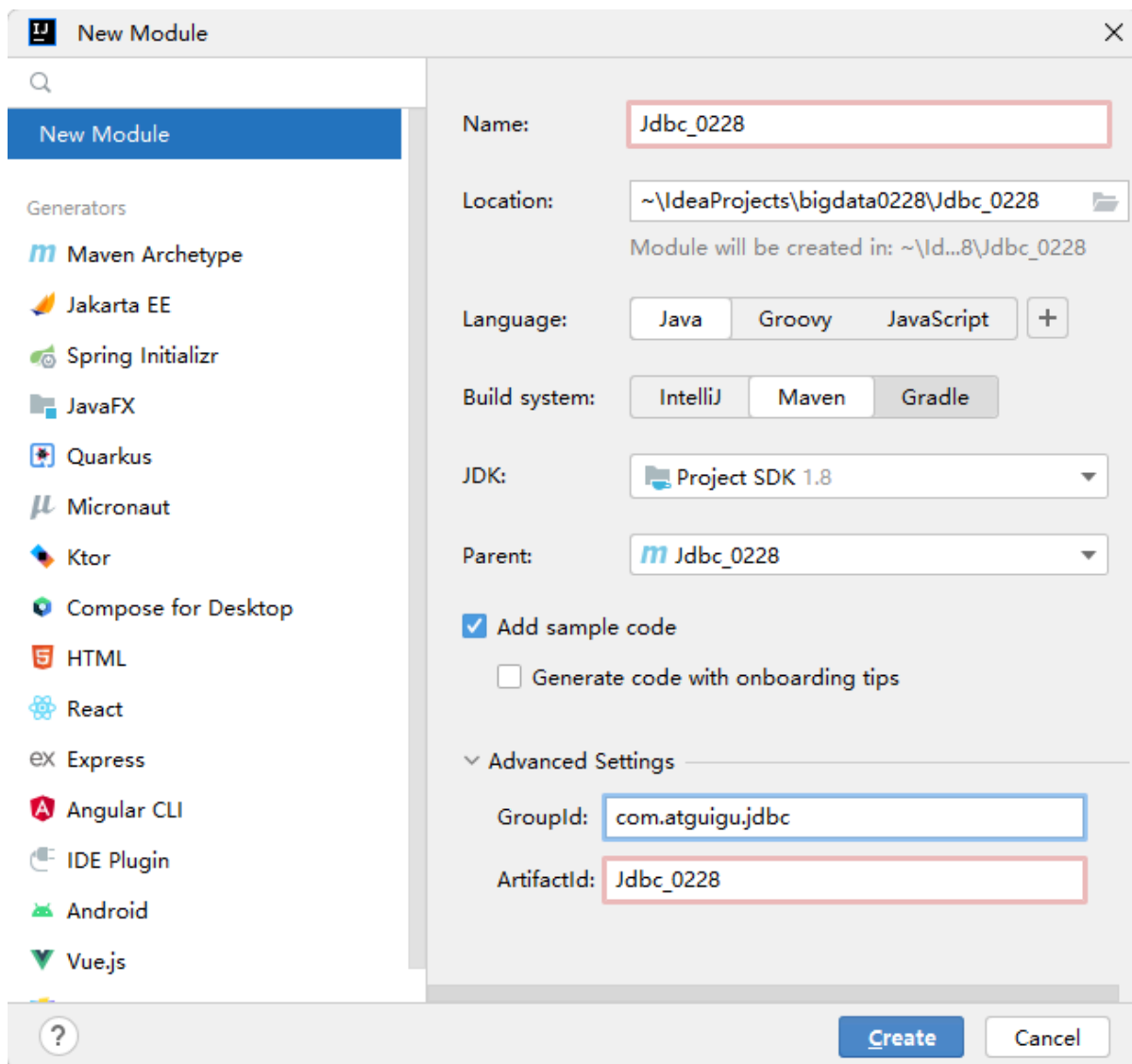
2.3 要素三：用户名和密码

- user,password可以用“属性名=属性值”方式告诉数据库
- 可以调用 DriverManager 类的 getConnection() 方法建立到数据库的连接

2.4 数据库连接方式举例

2.4.0 创建maven项目 Jdbc_0228 如下图：

2.4.0.1:创建项目



2.4.0.1:在pom.xml文件中导入以下依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.atguigu.jdbc</groupId>
    <artifactId>Jdbc_0228</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>8</maven.compiler.source>
        <maven.compiler.target>8</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <!--junit单元测试包-->
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.12</version>
        </dependency>
        <!--jdbc的mysql驱动包-->
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
            <version>8.0.26</version>
        </dependency>
    </dependencies>
</project>
```

2.4.1 连接方式一 (选学)

```
package com.atguigu.jdbc;

import com.mysql.cj.jdbc.Driver;
import org.junit.Test;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class JdbcConnectionTest {

    @Test
    public void testConnection1(){
        try {
            //1. 提供mysql中Driver接口的实现类
            Driver driver = new Driver();
            //2. 注册驱动
            DriverManager.registerDriver(driver);
```

```

        //3. 提供其它3个基本信息
        //url, 指明具体操作的数据。其中, test:表示具体的数据库名
        String url = "jdbc:mysql://hadoop100:3306/my_db?
useSSL=false&serverTimezone=UTC";
        String user = "root";
        String password = "000000";
        //4. 获取连接
        Connection connection = DriverManager.getConnection(url, user,
password);
        System.out.println(connection);
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
}

```

说明：上述代码中显式出现了第三方数据库的API

2.4.2 连接方式二 (选学)

```

@Test
public void testConnection2() throws Exception {

    //1. 提供mysql中Driver接口的实现类的对象
    String className = "com.mysql.cj.jdbc.Driver";
    Class clazz = Class.forName(className);
    Driver driver = (Driver) clazz.getDeclaredConstructor().newInstance();

    //2. 注册驱动
    DriverManager.registerDriver(driver);

    //3. 提供其它3个基本信息
    String url = "jdbc:mysql://hadoop100:3306/my_db?
useSSL=false&serverTimezone=UTC";
    String user = "root";
    String password = "000000";

    //4. 获取连接
    Connection connection = DriverManager.getConnection(url, user, password);
    System.out.println(connection);

}

```

说明：相较于方式一，这里使用反射实例化Driver，不在代码中体现第三方数据库的API。体现了面向接口编程思想，使程序具备更好的移植性

2.4.3 连接方式三 (选学)

```

@Test
public void testConnection3() throws Exception {

    //1. 获取连接的4个基本信息
    String className = "com.mysql.cj.jdbc.Driver";
    String url = "jdbc:mysql://hadoop100:3306/my_db?
useSSL=false&serverTimezone=UTC";

```



```

String user = "root";
String password = "000000";

//2. 加载驱动
Class.forName(className);

//注册驱动:略
//      DriverManager.registerDriver(driver);
/*
 * 之所以不再代码中显式使用DriverManager的注册功能, 是因为在mysql的Driver类的源码中
发现:
 * static {
 *     try {
 *         java.sql.DriverManager.registerDriver(new Driver());
 *     } catch (SQLException E) {
 *         throw new RuntimeException("Can't register driver!");
 *     }
 * }
 *
 * */
//3. 获取连接
Connection connection = DriverManager.getConnection(url, user, password);
System.out.println(connection);

}

```

说明: 不必显式的注册驱动了。因为在DriverManager的源码中已经存在静态代码块, 实现了驱动的注册。

2.4.4 连接方式四(最终版)

```

@Test
public void testConnection4() throws Exception {
    //1. 读取配置文件中的4个基本信息
    Properties pros = new Properties();
    //2. 加载资源的路径默认为 src/main/java/resources 目录下
    InputStream is =
    ClassLoader.getSystemClassLoader().getResourceAsStream("jdbc.properties");
    pros.load(is);

    String className = pros.getProperty("className");
    String url = pros.getProperty("url");
    String user = pros.getProperty("user");
    String password = pros.getProperty("password");

    //3. 加载驱动
    Class.forName(className);

    //4. 获取连接
    Connection connection = DriverManager.getConnection(url, user, password);
    System.out.println(connection);
}

```

其中, 配置文件声明在当前项目的src/java/resources目录下: 【jdbc.properties】

```
className=com.mysql.cj.jdbc.Driver
url=jdbc:mysql://hadoop100:3306/my_db?useSSL=false&serverTimezone=UTC
user=root
password=000000
```

说明：使用配置文件的方式保存配置信息，在代码中加载配置文件

使用配置文件的好处：

- ①实现了代码和数据的分离，如果需要修改配置信息，直接在配置文件中修改，不需要深入代码
- ②编写的java程序在部署到服务器上时，需要打包。如果java代码修改过，就需要重新打包。使用配置文件的方式，如果配置信息修改，并没有导致代码的修改。所以不需要重写打包。

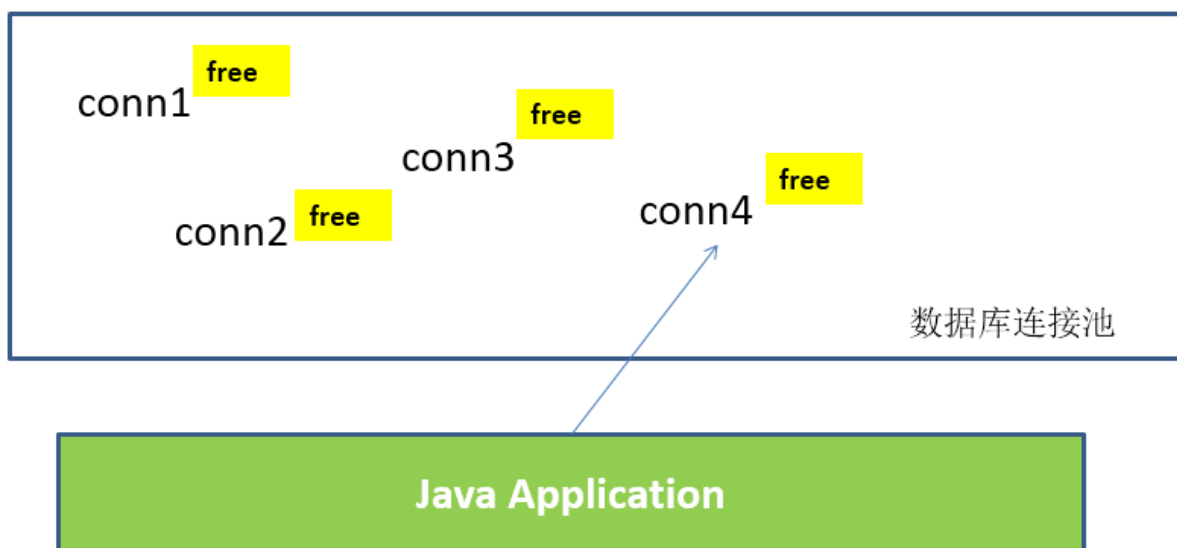
第3章：数据库连接池

3.1 JDBC数据库连接池的必要性

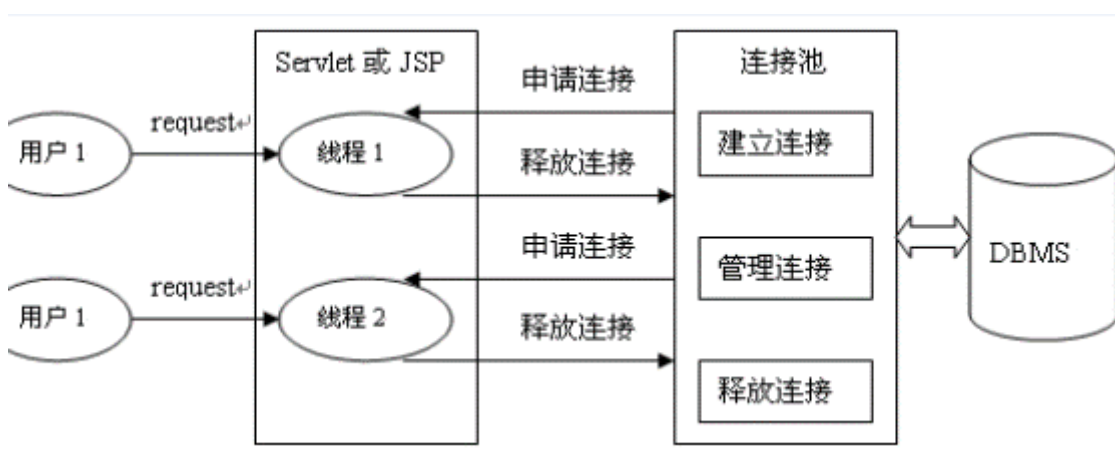
- 在使用开发基于数据库的web程序时，传统的模式基本是按以下步骤：
 - 在主程序（如servlet、beans）中建立数据库连接
 - 进行sql操作
 - 断开数据库连接
- 这种模式开发，存在的问题：
 - 普通的JDBC数据库连接使用 DriverManager 来获取，每次向数据库建立连接的时候都要将 Connection 加载到内存中，再验证用户名和密码(得花费0.05s ~ 1s的时间)。需要数据库连接的时候，就向数据库要求一个，执行完成后断开连接。这样的方式将会消耗大量的资源和时间。**数据库的连接资源并没有得到很好的重复利用。**若同时有几百人甚至几千人在线，频繁的数据库连接操作将占用很多的系统资源，严重的甚至会造成服务器的崩溃。
 - **对于每一次数据库连接，使用完后都得断开。**否则，如果程序出现异常而未能关闭，将会导致数据库系统中的内存泄漏，最终将导致重启数据库。（回忆：何为Java的内存泄漏？）
 - **这种开发不能控制被创建的连接对象数**，系统资源会被毫无顾及的分配出去，如连接过多，也可能导致服务器崩溃。

3.2 数据库连接池技术

- 为解决传统开发中的数据库连接问题，可以采用数据库连接池技术。
- **数据库连接池的基本思想：**就是为数据库连接建立一个“缓冲池”。预先在缓冲池中放入一定数量的连接，当需要建立数据库连接时，只需从“缓冲池”中取出一个，使用完毕之后再放回去。
- **数据库连接池负责分配、管理和释放数据库连接，它允许应用程序重复使用一个现有的数据库连接，而不是重新建立一个。**
- 数据库连接池在初始化时将创建一定数量的数据库连接放到连接池中，这些数据库连接的数量是由**最小数据库连接数**来设定的。无论这些数据库连接是否被使用，连接池都将一直保证至少拥有这么多的连接数量。连接池的**最大数据库连接数量**限定了这个连接池能占有的最大连接数，当应用程序向连接池请求的连接数超过最大连接数量时，这些请求将被加入到等待队列中。



• 工作原理:



• 数据库连接池技术的优点

1. 资源重用

由于数据库连接得以重用，避免了频繁创建，释放连接引起的大量性能开销。在减少系统消耗的基础上，另一方面也增加了系统运行环境的平稳性。

2. 更快的系统反应速度

数据库连接池在初始化过程中，往往已经创建了若干数据库连接置于连接池中备用。此时连接的初始化工作均已完成。对于业务请求处理而言，直接利用现有可用连接，避免了数据库连接初始化和释放过程的时间开销，从而减少了系统的响应时间。

3. 统一的连接管理，避免数据库连接泄漏

在较为完善的数据库连接池实现中，可根据预先的占用超时设定，强制回收被占用连接，从而避免了常规数据库连接操作中可能出现的资源泄漏。

4. 新的资源分配手段

对于多应用共享同一数据库的系统而言，可在应用层通过数据库连接池的配置，实现某一应用最大可用数据库连接数的限制，避免某一应用独占所有的数据库资源。

3.3 多种开源的数据库连接池

- JDBC 的数据库连接池使用 `javax.sql.DataSource` 来表示，`DataSource` 只是一个接口，该接口通常由服务器(Weblogic, WebSphere, Tomcat)提供实现，也有一些开源组织提供实现：
 - **DBCP** 是Apache提供的数据库连接池。tomcat 服务器自带dbcp数据库连接池。**速度相对c3p0较快**，但因自身存在BUG，Hibernate3已不再提供支持。

- **C3P0** 是一个开源组织提供的一个数据库连接池，**速度相对较慢，稳定性还可以**。hibernate官方推荐使用
- **Proxool** 是sourceforge下的一个开源项目数据库连接池，有监控连接池状态的功能，**稳定性较c3p0差一点**
- **BoneCP** 是一个开源组织提供的数据库连接池，速度快
- **Druid** 是阿里提供的数据库连接池，据说是集DBCP、C3P0、Proxool 优点于一身的数据库连接池，但是速度不确定是否有BoneCP快
- DataSource 通常被称为数据源，它包含连接池和连接池管理两个部分，习惯上也经常把DataSource 称为连接池
- **DataSource用来取代DriverManager来获取Connection，获取速度快，同时可以大幅度提高数据库访问速度。**
- 特别注意：
 - 数据源和数据库连接不同，数据源无需创建多个，它是产生数据库连接的工厂，因此**整个应用只需要一个数据源即可**。
 - 当数据库访问结束后，程序还是像以前一样关闭数据库连接：conn.close(); 但conn.close()并没有关闭数据库的物理连接，它仅仅把数据库连接释放，归还给了数据库连接池。

3.4 Druid（德鲁伊）数据库连接池

Druid是阿里巴巴开源平台上一个数据库连接池实现，它结合了C3P0、DBCP、Proxool等DB池的优点，同时加入了日志监控，可以很好的监控DB池连接和SQL的执行情况，可以说是针对监控而生的DB连接池，**可以说是目前最好的连接池之一**。

方式一：

```
package com.atguigu.jdbc;
import com.alibaba.druid.pool.DruidDataSource;
import org.junit.Test;

import java.sql.Connection;

import java.sql.SQLException;

public class JdbcDruidConnectionTest {

    @Test
    public void testDruidConnection1() {
        //看做是获取了一个数据库连接池
        DruidDataSource source = new DruidDataSource();

        //设置4个基本信息
        source.setUsername("root");
        source.setPassword("000000");
        source.setUrl("jdbc:mysql://hadoop108:3306/my_db?
useSSL=false&serverTimezone=UTC");
        source.setDriverClassName("com.mysql.cj.jdbc.Driver");

        //还可以设置其他信息
        source.setMaxActive(10);
    }
}
```

```

        source.setInitialSize(5);

        //获取连接池中的一个数据库连接
        Connection connection = null;
        try {
            connection = source.getConnection();
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }

        System.out.println(connection);
    }
}

```

方式二：使用配置文件

```

@Test
public void testDruidConnection2() throws Exception {
    //提供Properties，并加载指定配置文件的流
    Properties pros = new Properties();
    InputStream is =
        ClassLoader.getSystemClassLoader().getResourceAsStream("druid.properties");
    pros.load(is);

    //通过DruidDataSourceFactory创建一个数据源
    DataSource dataSource = DruidDataSourceFactory.createDataSource(pros);

    //通过数据源获取连接
    System.out.println(dataSource.getConnection());
}

```

其中，src/main/java/resources下的配置文件为：【druid.properties】

```

driverClassName=com.mysql.cj.jdbc.Driver
url=jdbc:mysql://hadoop100:3306/test?useSSL=false&serverTimezone=UTC
username=root
password=000000

maxActive=10
initialSize=5

```

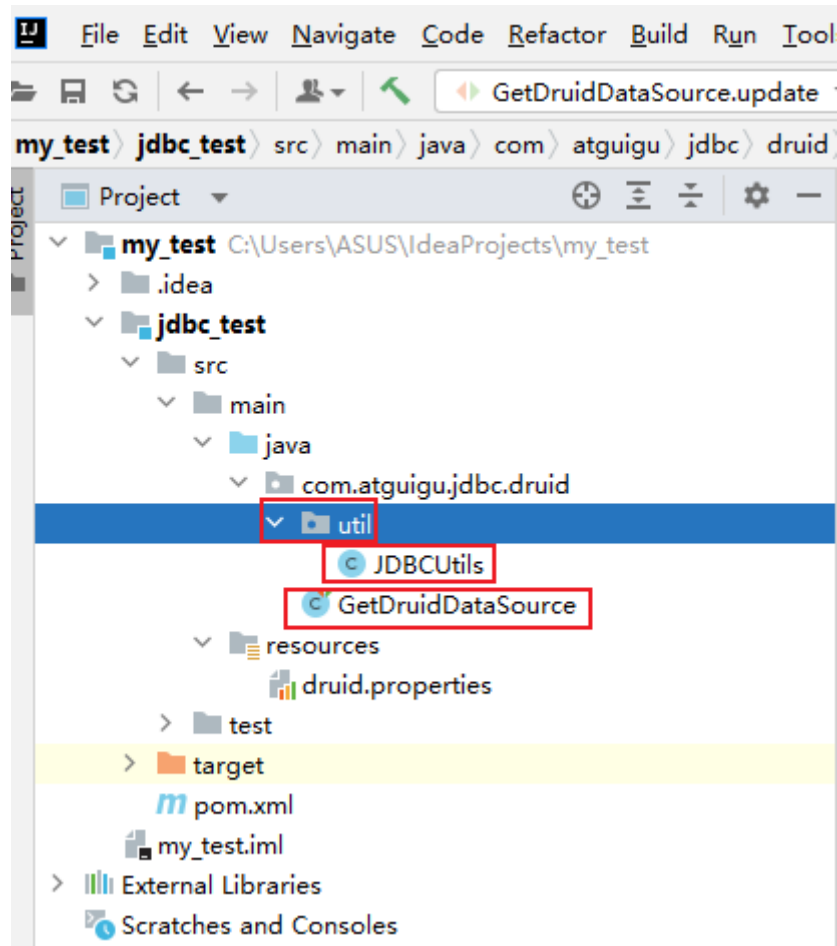
- 详细配置参数：

配置	缺省	说明
username		连接数据库的用户名
password		连接数据库的密码。如果你不希望密码直接写在配置文件中，可以使用ConfigFilter。详细看这里： https://github.com/alibaba/druid/wiki/%E4%BD%BF%E7%94%A8ConfigFilter

配置	缺省	说明
url		连接数据库的url，不同数据库不一样。例如：mysql： jdbc:mysql://10.20.153.104:3306/druid2 oracle： jdbc:oracle:thin:@10.20.149.85:1521:ocnauto
driverClassName		根据url自动识别 这一项可配可不配，如果不配置druid会根据url自动识别dbType，然后选择相应的driverClassName(建议配置下)
name		配置这个属性的意义在于，如果存在多个数据源，监控的时候可以通过名字来区分开来。如果没有配置，将会生成一个名字，格式是：“DataSource-” + System.identityHashCode(this)
initialSize	0	初始化时建立物理连接的个数。初始化发生在显示调用init方法，或者第一次getConnection时
maxActive	8	最大连接池数量
maxIdle	8	已经不再使用，配置了也没效果
minIdle		最小连接池数量
maxWait		获取连接时最大等待时间，单位毫秒。配置了maxWait之后，缺省启用公平锁，并发效率会有所下降，如果需要可以通过配置useUnfairLock属性为true使用非公平锁。
poolPreparedStatements	false	是否缓存preparedStatement，也就是PSCache。PSCache对支持游标的数据库性能提升巨大，比如说oracle。在mysql下建议关闭。
maxOpenPreparedStatements	-1	要启用PSCache，必须配置大于0，当大于0时，poolPreparedStatements自动触发修改为true。在Druid中，不会存在Oracle下PSCache占用内存过多的问题，可以把这个数值配置大一些，比如说100
validationQuery		用来检测连接是否有效的sql，要求是一个查询语句。如果validationQuery为null，testOnBorrow、testOnReturn、testWhileIdle都不会起作用。
testOnBorrow	true	申请连接时执行validationQuery检测连接是否有效，做了这个配置会降低性能。
testOnReturn	false	归还连接时执行validationQuery检测连接是否有效，做了这个配置会降低性能
testWhileIdle	false	建议配置为true，不影响性能，并且保证安全性。申请连接的时候检测，如果空闲时间大于timeBetweenEvictionRunsMillis，执行validationQuery检测连接是否有效。
timeBetweenEvictionRunsMillis		有两个含义： 1)Destroy线程会检测连接的间隔时间 2)testWhileIdle的判断依据，详细看testWhileIdle属性的说明
numTestsPerEvictionRun		不再使用，一个DruidDataSource只支持一个EvictionRun

配置	缺省	说明
minEvictableIdleTimeMillis		
connectionInitSqls		物理连接初始化的时候执行的sql
exceptionSorter		根据dbType自动识别 当数据库抛出一些不可恢复的异常时，抛弃连接
filters		属性类型是字符串，通过别名的方式配置扩展插件，常用的插件有： 监控统计用的filter:stat日志用的filter:log4j防御sql注入的filter:wall
proxyFilters		类型是List，如果同时配置了filters和proxyFilters，是组合关系，并非替换关系

3.4 封装JDBCUtils



GetDruidDataSource 类

```
package com.atguigu.jdbc.druid;

import com.alibaba.druid.pool.DruidDataSource;
import com.alibaba.druid.pool.DruidDataSourceFactory;
import com.atguigu.jdbc.druid.util.JDBCUtils;
import org.junit.Test;
```



```

import javax.sql.DataSource;
import java.io.IOException;
import java.io.InputStream;
import java.sql.Connection;
import java.util.Properties;

public class GetDruidDataSource {

    /**
     * 测试工具类获取连接
     * @throws Exception
     */
    @Test
    public void update() throws Exception {
        // 获取连接对象
        Connection connection = JDBCUtils.getConnection();
        System.out.println(connection);

        // 进行数据的crud

        // 释放连接
        JDBCUtils.close(connection);
    }
}

```

JDBCUtils类

```

package com.atguigu.jdbc.druid.util;

import com.alibaba.druid.pool.DruidDataSourceFactory;

import javax.sql.DataSource;
import java.io.IOException;
import java.io.InputStream;
import java.sql.Connection;
import java.sql.SQLException;
import java.util.Properties;

public class JDBCUtils {
    // 声明DataSource对象
    private static DataSource dataSource = null;

    // 静态代码块：用于类加载时就获取连接池对象
    static {
        // 创建Properties对象
        Properties properties = new Properties();
        InputStream inputStream = ClassLoader.getSystemClassLoader()
            .getResourceAsStream("druid.properties");
        try {
            properties.load(inputStream);
            // 通过DruidDataSourceFactory对象获取数据源对象
            dataSource = DruidDataSourceFactory.createDataSource(properties);
        } catch (IOException e) {

```

```

        throw new RuntimeException(e);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}

// 封装获取连接的方法
public static Connection getConnection() throws Exception {
    Connection connection = dataSource.getConnection();
    return connection;
}

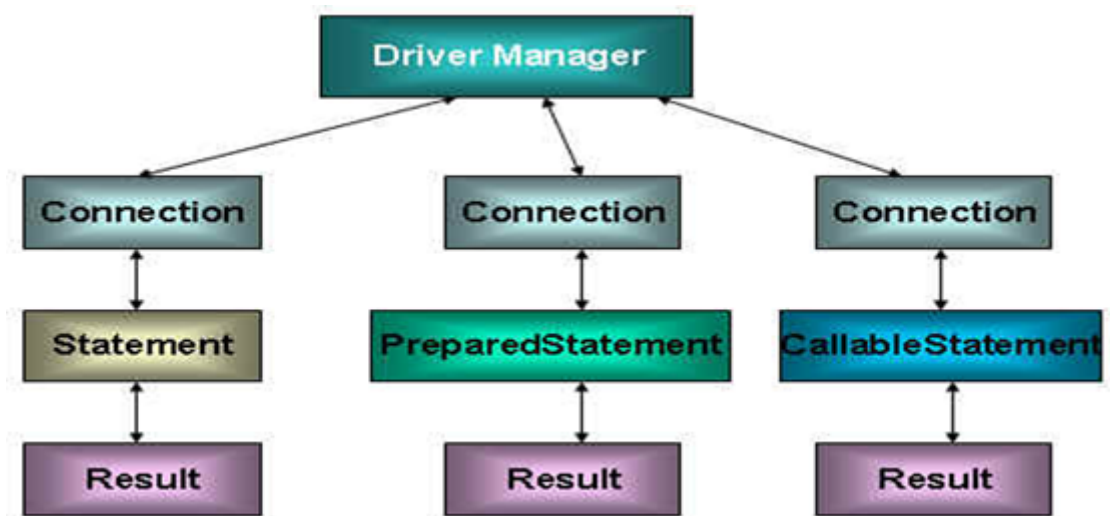
// 关闭连接的方法
public static void close(Connection connection) throws SQLException {
    if (connection != null) {
        connection.close();
    }
}
}

```

第4章：使用PreparedStatement实现CRUD操作（了解）

4.1 操作和访问数据库

- 数据库连接被用于向数据库服务器发送命令和 SQL 语句，并接受数据库服务器返回的结果。其实一个数据库连接就是一个Socket连接。
- 在 java.sql 包中有 3 个接口分别定义了对数据库的调用的不同方式：
 - Statement：用于执行静态 SQL 语句并返回它所生成结果的对象。
 - PreparedStatement：SQL 语句被预编译并存储在此对象中，可以使用此对象多次高效地执行该语句。
 - CallableStatement：用于执行 SQL 存储过程



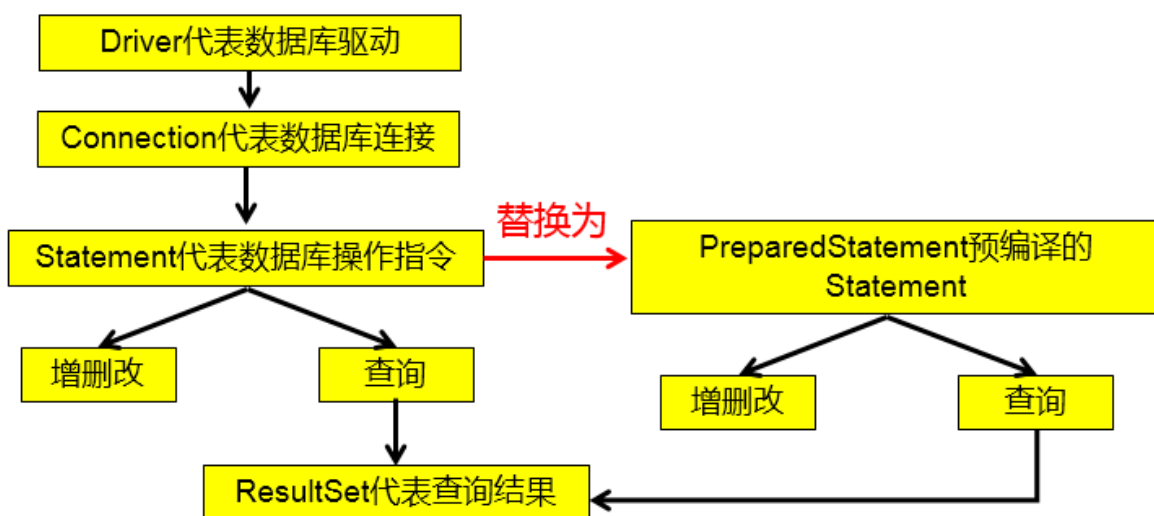
4.2 使用Statement操作数据表的弊端（了解）

- 通过调用 Connection 对象的 createStatement() 方法创建该对象。该对象用于执行静态的 SQL 语句，并且返回执行结果。
- Statement 接口中定义了下列方法用于执行 SQL 语句：

```
int executeUpdate(String sql): 执行更新操作INSERT、UPDATE、DELETE  
ResultSet executeQuery(String sql): 执行查询操作SELECT
```

- 但是使用Statement操作数据表存在弊端：
 - 问题一：存在拼串操作，繁琐
 - 问题二：存在SQL注入问题
- SQL 注入是利用某些系统没有对用户输入的数据进行充分的检查，而在用户输入数据中注入非法的 SQL 语句段或命令(如：SELECT user, password FROM user_table WHERE user='a' OR 1 = ' AND password = ' OR '1' = '1')，从而利用系统的 SQL 引擎完成恶意行为的做法。
- 对于 Java 而言，要防范 SQL 注入，只要用 PreparedStatement(从Statement扩展而来) 取代 Statement 就可以了。
- 代码演示（略）：

综上：



4.3 PreparedStatement的使用

4.3.1 PreparedStatement介绍

- 可以通过调用 Connection 对象的 **prepareStatement(String sql)** 方法获取 PreparedStatement 对象
- PreparedStatement 接口是 Statement 的子接口，它表示一条预编译过的 SQL 语句**
- PreparedStatement 对象所代表的 SQL 语句中的参数用问号(?)来表示，调用 PreparedStatement 对象的 setXxx() 方法来设置这些参数. setXxx() 方法有两个参数，第一个参数是要设置的 SQL 语句中的参数的索引(从 1 开始)，第二个是设置的 SQL 语句中的参数的值

4.3.2 PreparedStatement vs Statement

- 代码的可读性和可维护性。
- **PreparedStatement 能最大可能提高性能：**
 - DBServer会对**预编译**语句提供性能优化。因为预编译语句有可能被重复调用，所以语句在被DBServer的编译器编译后的执行代码被缓存下来，那么下次调用时只要是相同的预编译语句就不需要编译，只要将参数直接传入编译过的语句执行代码中就会得到执行。
 - 在Statement语句中,即使是相同操作但因为数据内容不一样,所以整个语句本身不能匹配,没有缓存语句的意义.事实是没有数据库会对普通语句编译后的执行代码缓存。这样每执行一次都要对传入的语句编译一次。
 - (语法检查，语义检查，翻译成二进制命令，缓存)
- PreparedStatement 可以防止 SQL 注入

4.3.3 Java与SQL对应数据类型转换表

Java类型	SQL类型
boolean	BIT
byte	TINYINT
short	SMALLINT
int	INTEGER
long	BIGINT
String	CHAR,VARCHAR,LONGVARCHAR
byte array	BINARY , VAR BINARY
java.sql.Date	DATE
java.sql.Time	TIME
java.sql.Timestamp	TIMESTAMP

4.3.4 使用PreparedStatement实现增、删、改操作

```
//通用的增、删、改操作（体现一：增、删、改； 体现二：针对于不同的表）
public void update(String sql,Object ... args){
    Connection conn = null;
    PreparedStatement ps = null;
    try {
        //1.获取数据库的连接
        conn = JDBCUtils.getConnection();

        //2.获取PreparedStatement的实例（或：预编译sql语句）
        ps = conn.prepareStatement(sql);
        //3.填充占位符
        for(int i = 0;i < args.length;i++){
            ps.setObject(i + 1, args[i]);
        }
    }
```

```

        //4.执行sql语句
        ps.execute();
    } catch (Exception e) {

        e.printStackTrace();
    } finally {
        //5.关闭资源
        JDBCUtils.closeResource(conn, ps);
    }
}

```

4.3.5 使用PreparedStatement实现查询操作

```

// 通用的针对于不同表的查询:返回一个对象 (version 1.0)
public <T> T getInstance(Class<T> clazz, String sql, Object... args) {

    Connection conn = null;
    PreparedStatement ps = null;
    ResultSet rs = null;
    try {
        // 1.获取数据库连接
        conn = JDBCUtils.getConnection();

        // 2.预编译sql语句,得到PreparedStatement对象
        ps = conn.prepareStatement(sql);

        // 3.填充占位符
        for (int i = 0; i < args.length; i++) {
            ps.setObject(i + 1, args[i]);
        }

        // 4.执行executeQuery(),得到结果集: ResultSet
        rs = ps.executeQuery();

        // 5.得到结果集的元数据: ResultSetMetaData
        ResultSetMetaData rsmd = rs.getMetaData();

        // 6.1通过ResultSetMetaData得到columnCount,columnLabel; 通过ResultSet得到列值
        int columnCount = rsmd.getColumnCount();
        if (rs.next()) {
            T t = clazz.newInstance();
            for (int i = 0; i < columnCount; i++) { // 遍历每一个列

                // 获取列值
                Object columnVal = rs.getObject(i + 1);
                // 获取列的别名:列的别名,使用类的属性名充当
                String columnLabel = rsmd.getColumnLabel(i + 1);
                // 6.2使用反射,给对象的相应属性赋值
                Field field = clazz.getDeclaredField(columnLabel);
                field.setAccessible(true);
                field.set(t, columnVal);
            }

            return t;
        }
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        JDBCUtils.closeResource(conn, ps, rs);
    }
}

```

```

    }
} catch (Exception e) {

    e.printStackTrace();
} finally {
    // 7.关闭资源
    JDBCUtils.closeResource(conn, ps, rs);
}

return null;
}

```

说明：使用PreparedStatement实现的查询操作可以替换Statement实现的查询操作，解决Statement拼串和SQL注入问题。

4.5 资源的释放

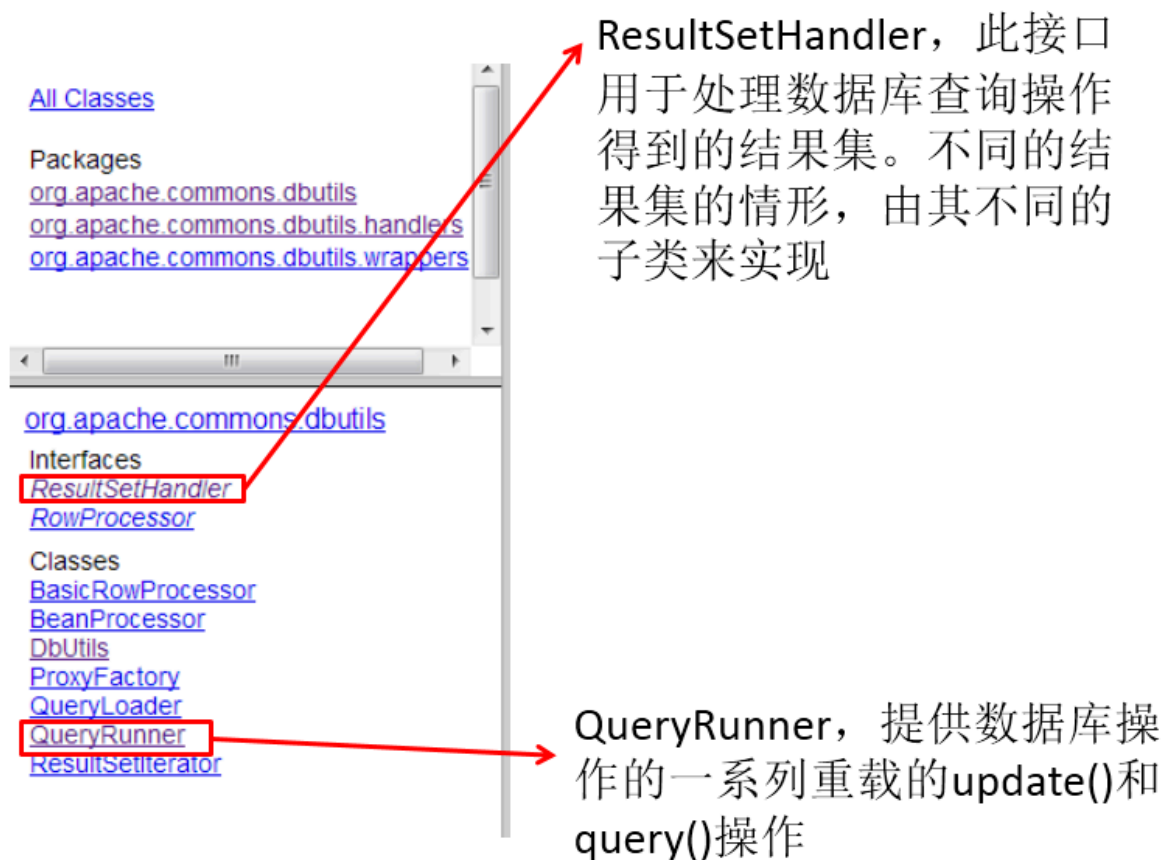
- 释放ResultSet, Statement, Connection。
- 数据库连接（Connection）是非常稀有的资源，用完后必须马上释放，如果Connection不能及时正确的关闭将导致系统宕机。Connection的使用原则是**尽量晚创建，尽量早的释放**。
- 可以在finally中关闭，保证及时其他代码出现异常，资源也一定能被关闭。

第5章：Apache-DBUtils实现CRUD操作

5.1 Apache-DBUtils简介

- commons-dbutils 是 Apache 组织提供的一个开源 JDBC工具类库，它是对JDBC的简单封装，学习成本极低，并且使用dbutils能极大简化jdbc编码的工作量，同时也不会影响程序的性能。
- API介绍：
 - org.apache.commons.dbutils.QueryRunner
 - org.apache.commons.dbutils.ResultSetHandler
 - 工具类：org.apache.commons.dbutils.DbUtils
- API包说明：





5.2 主要API的使用

在项目的pom.xml文件中导入一下依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.atguigu.jdbc</groupId>
  <artifactId>jdbc_test</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>commons-dbutils</groupId>
      <artifactId>commons-dbutils</artifactId>
      <version>1.3</version>
    </dependency>
  </dependencies>

</project>
```


5.2.1 DbUtils

- DbUtils：提供如关闭连接、装载JDBC驱动程序等常规工作的工具类，里面的所有方法都是静态的。主要方法如下：
 - **public static void close(...) throws java.sql.SQLException**： DbUtils类提供了三个重载的关闭方法。这些方法检查所提供的参数是不是NULL，如果不是的话，它们就关闭Connection、Statement和ResultSet。
 - **public static void closeQuietly(...)**: 这一类方法不仅能在Connection、Statement和ResultSet为NULL情况下避免关闭，还能隐藏一些在程序中抛出的SQLException。
 - **public static void commitAndClose(Connection conn) throws SQLException**： 用来提交连接的事务，然后关闭连接
 - **public static void commitAndCloseQuietly(Connection conn)**： 用来提交连接，然后关闭连接，并且在关闭连接时不抛出SQL异常。
 - **public static void rollback(Connection conn) throws SQLException**： 允许conn为null，因为方法内部做了判断
 - **public static void rollbackAndClose(Connection conn) throws SQLException**
 - **rollbackAndCloseQuietly(Connection)**
 - **public static boolean loadDriver(java.lang.String driverClassName)**： 这一方装载并注册JDBC驱动程序，如果成功就返回true。使用该方法，你不需要捕捉这个异常ClassNotFoundException。

5.2.2 QueryRunner类

- 该类简化了SQL查询，它与ResultSetHandler组合在一起使用可以完成大部分的数据库操作，能够大大减少编码量。
- QueryRunner类提供了两个构造器：
 - 默认的构造器
 - 需要一个 javax.sql.DataSource 来作参数的构造器
- QueryRunner类的主要方法：
 - **更新**
 - **public int update(Connection conn, String sql, Object... params) throws SQLException**: 用来执行一个更新（插入、更新或删除）操作。
 -
 - **插入**
 - **public T insert(Connection conn, String sql, ResultSetHandler rsh, Object... params) throws SQLException**： 只支持INSERT语句，其中 rsh - The handler used to create the result object from the ResultSet of auto-generated keys. 返回值: An object generated by the handler.即自动生成的键值
 -
 - **批处理**
 - **public int[] batch(Connection conn, String sql, Object[][] params) throws SQLException**： INSERT, UPDATE, or DELETE语句
 - **public T insertBatch(Connection conn, String sql, ResultSetHandler rsh, Object[][] params) throws SQLException**： 只支持INSERT语句

-
- 查询
 - public Object query(Connection conn, String sql, ResultSetHandler rsh, Object... params) throws SQLException: 执行一个查询操作, 在这个查询中, 对象数组中的每个元素值被用来作为查询语句的置换参数。该方法会自行处理 PreparedStatement 和 ResultSet 的创建和关闭。
 -

- 测试

```
// 测试添加
@Test
public void testInsert() throws Exception {
    QueryRunner runner = new QueryRunner();
    Connection conn = JDBCUtils.getConnection3();
    String sql = "insert into customers(name,email,birth)values(?,?,?)";
    int count = runner.update(conn, sql, "何成飞", "he@qq.com", "1992-09-08");

    System.out.println("添加了" + count + "条记录");

    JDBCUtils.closeResource(conn, null);
}
```

```
// 测试删除
@Test
public void testDelete() throws Exception {
    QueryRunner runner = new QueryRunner();
    Connection conn = JDBCUtils.getConnection3();
    String sql = "delete from customers where id < ?";
    int count = runner.update(conn, sql, 3);

    System.out.println("删除了" + count + "条记录");

    JDBCUtils.closeResource(conn, null);
}
```

5.2.3 ResultSetHandler接口及实现类

- 该接口用于处理 java.sql.ResultSet, 将数据按要求转换为另一种形式。
- ResultSetHandler 接口提供了一个单独的方法: Object handle (java.sql.ResultSet .rs)。
- 接口的主要实现类:
 - ArrayHandler: 把结果集中的第一行数据转成对象数组。
 - ArrayListHandler: 把结果集中的每一行数据都转成一个数组, 再存放到List中。
 - **BeanHandler**: 将结果集中的第一行数据封装到一个对应的JavaBean实例中。

- **BeanListHandler**: 将结果集中的每一行数据都封装到一个对应的JavaBean实例中, 存放到List里。
 - **ColumnListHandler**: 将结果集中某一列的数据存放到List中。
 - **KeyedHandler(name)**: 将结果集中的每一行数据都封装到一个Map里, 再把这些map再存到一个map里, 其key为指定的key。
 - **MapHandler**: 将结果集中的第一行数据封装到一个Map里, key是列名, value就是对应的值。
 - **MapListHandler**: 将结果集中的每一行数据都封装到一个Map里, 然后再存放到List
 - **ScalarHandler**: 查询单个值对象
- 测试

```

/*
 * 测试查询: 查询一条记录
 *
 * 使用ResultSetHandler的实现类: BeanHandler
 */
@Test
public void testQueryInstance() throws Exception{
    QueryRunner runner = new QueryRunner();

    Connection conn = JDBCUtils.getConnection3();

    String sql = "select id,name,email,birth from customers where id = ?";

    //
    BeanHandler<Customer> handler = new BeanHandler<>(Customer.class);
    Customer customer = runner.query(conn, sql, handler, 23);
    System.out.println(customer);
    JDBCUtils.closeResource(conn, null);
}

```

```

/*
 * 测试查询: 查询多条记录构成的集合
 *
 * 使用ResultSetHandler的实现类: BeanListHandler
 */
@Test
public void testQueryList() throws Exception{
    QueryRunner runner = new QueryRunner();

    Connection conn = JDBCUtils.getConnection3();

    String sql = "select id,name,email,birth from customers where id < ?";

    //
    BeanListHandler<Customer> handler = new BeanListHandler<>(Customer.class);
    List<Customer> list = runner.query(conn, sql, handler, 23);
    list.forEach(System.out::println);

    JDBCUtils.closeResource(conn, null);
}

```

```
}
```

```
/*
 * 自定义ResultSetHandler的实现类
 */
@Test
public void testQueryInstance1() throws Exception{
    QueryRunner runner = new QueryRunner();

    Connection conn = JDBCUtils.getConnection3();

    String sql = "select id,name,email,birth from customers where id = ?";

    ResultSetHandler<Customer> handler = new ResultSetHandler<Customer>() {

        @Override
        public Customer handle(ResultSet rs) throws SQLException {
            System.out.println("handle");
//            return new Customer(1,"Tom","tom@126.com",new Date(123323432L));

            if(rs.next()){
                int id = rs.getInt("id");
                String name = rs.getString("name");
                String email = rs.getString("email");
                Date birth = rs.getDate("birth");

                return new Customer(id, name, email, birth);
            }
            return null;
        }
    };

    Customer customer = runner.query(conn, sql, handler, 23);

    System.out.println(customer);

    JDBCUtils.closeResource(conn, null);
}
```

```
/*
 * 如何查询类似于最大的，最小的，平均的，总和，个数相关的数据，
 * 使用ScalarHandler
 *
 */
@Test
public void testQueryValue() throws Exception{
    QueryRunner runner = new QueryRunner();

    Connection conn = JDBCUtils.getConnection3();

    //测试一：
    // String sql = "select count(*) from customers where id < ?";
    // ScalarHandler handler = new ScalarHandler();
}
```

```
// long count = (long) runner.query(conn, sql, handler, 20);
// System.out.println(count);

//测试二：
String sql = "select max(birth) from customers";
ScalarHandler handler = new ScalarHandler();
Date birth = (Date) runner.query(conn, sql, handler);
System.out.println(birth);

JDBCUtils.closeResource(conn, null);
}
```

第6章：DAO及相关实现类

- DAO：Data Access Object访问数据信息的类和接口，包括了对数据的CRUD（Create、Retrival、Update、Delete），而不包含任何业务相关的信息。有时也称作：BaseDAO
- 作用：为了实现功能的模块化，更有利于代码的维护和升级。
- 层次结构：站在项目整体结构角度，先得有通用的DAO层，能够满足各个表的通用crud操作，其次在根据不同的模块进行面向接口编程，最后在对接口进行实现，实现的过程中需要访问数据库进行crud就可以直接访问DAO层准备好的crud操作。

【DAO.java】

```
package com.atguigu.jdbc.dao;

import org.apache.commons.dbutils.QueryRunner;
import org.apache.commons.dbutils.handlers.BeanHandler;
import org.apache.commons.dbutils.handlers.BeanListHandler;
import org.apache.commons.dbutils.handlers.ScalarHandler;

import java.sql.Connection;
import java.sql.SQLException;
import java.util.List;

/**
 * 通用的增删改查操作
 */
public class DAO {

    private QueryRunner runner = new QueryRunner();

    /**
     * 通用的增删改操作
     */
    public int update(Connection conn, String sql, Object ... objs) throws
SQLException {
        return runner.update(conn, sql, objs);
    }

    /**
     * 查询一条记录，封装为一个对象
     */
}
```

```

    public <T> T getInstance(Connection conn,String sql,Class<T> clazz,Object ...
objs) throws SQLException {

        BeanHandler<T> handler = new BeanHandler<>(clazz);
        return runner.query(conn,sql,handler,objs);

    }

    /*
    * 返回多条记录, 封装为一个List
    * */
    public <T> List<T> getForList(Connection conn,String sql,Class<T>
clazz,Object ... objs) throws SQLException {
        BeanListHandler<T> handler = new BeanListHandler<>(clazz);
        return runner.query(conn,sql,handler,objs);
    }

    /*
    * 查询特殊值
    * */
    public <T> T getValue(Connection conn,String sql,Object ... objs) throws
SQLException {
        ScalarHandler handler = new ScalarHandler();
        return (T) runner.query(conn,sql,handler,objs);
    }

}

```

【Customer.java】

```

package com.atguigu.jdbc.dao;

import java.sql.Date;

/**
 * Customer实体类: 对应数据库的customer表
 */
public class Customer {
    private int id;
    private String name;
    private String email;
    private Date birth;

    public Customer() {
    }

    public Customer(int id, String name, String email, Date birth) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.birth = birth;
    }
}

```

```

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public Date getBirth() {
        return birth;
    }

    public void setBirth(Date birth) {
        this.birth = birth;
    }

    @Override
    public String toString() {
        return "Customer{" +
            "id=" + id +
            ", name='" + name + '\'' +
            ", email='" + email + '\'' +
            ", birth=" + birth +
            '}';
    }
}

```

【CustomerDAO.java】

```

package com.atguigu.jdbc.dao;
import java.sql.Connection;
import java.sql.Date;
import java.sql.SQLException;
import java.util.List;
public interface CustomerDAO {

```

```

/**
 * 将cust添加到指定的数据库中
 * @param conn 数据库连接
 * @param cust 要添加的对象
 * @return 添加的条目数
 */
int addCustomer(Connection conn, Customer cust) throws SQLException;

/**
 * 删除指定id的表中的数据
 * @param conn 数据库连接
 * @param id 指明要输出的数据的id
 * @return 删除的条目数
 */
int deleteById(Connection conn,int id) throws SQLException;

/**
 * update customers set name = ?,email = ?,birth = ? where id = ?
 * Customer cust = new Customer(10,"Tom","tom@126.com",new Date(234234L));
 *
 * 修改指定id位置上的customer信息
 * @param conn 数据库连接
 * @param cust 要修改及修改为的对象
 * @return 修改的条目数
 */
int updateCustomer(Connection conn, Customer cust) throws SQLException;

/**
 * 查询指定id的客户对象
 * @param conn 数据库连接
 * @param id 要查询对象的id值
 * @return 指定的id的对象
 */
Customer getCustomer(Connection conn,int id) throws SQLException;

/**
 * 查询customers表中的所有的记录
 * @param conn 数据库连接
 * @return 所有的客户对象构成的list
 */
List<Customer> getAll(Connection conn) throws SQLException;

/**
 * 查询customers表中的总条目数
 * @param conn 数据库连接
 * @return 总条目数
 */
long getCount(Connection conn) throws SQLException;

/**
 * 查询最大的生日
 * @param conn 数据库连接
 * @return 最大的生日
 */
Date getMaxBirth(Connection conn) throws SQLException;
}

```


【CustomerDAOImpl.java】

```
package com.atguigu.jdbc.dao;
import java.sql.Connection;
import java.sql.Date;
import java.sql.SQLException;
import java.util.List;
public class CustomerDAOImpl extends DAO implements CustomerDAO{

    @Override
    public int addCustomer(Connection conn, Customer cust) throws SQLException {
        String sql = "insert into customers(name,email,birth)values(?,?,?)";
        return update(conn,sql,cust.getName(),cust.getEmail(),cust.getBirth());
    }

    @Override
    public int deleteById(Connection conn, int id) throws SQLException {
        String sql = "delete from customers where id = ?";
        return update(conn,sql,id);
    }

    @Override
    public int updateCustomer(Connection conn, Customer cust) throws SQLException
    {
        String sql = "update customers set name = ?,email = ?,birth = ? where id
= ?";
        return
update(conn,sql,cust.getName(),cust.getEmail(),cust.getBirth(),cust.getId());
    }

    @Override
    public Customer getCustomer(Connection conn, int id) throws SQLException {
        String sql = "select id,name,email,birth from customers where id = ?";
        return getInstance(conn,sql,Customer.class,id);
    }

    @Override
    public List<Customer> getAll(Connection conn) throws SQLException {
        String sql = "select id,name,email,birth from customers";
        return getForList(conn,sql,Customer.class);
    }

    @Override
    public long getCount(Connection conn) throws SQLException {
        String sql = "select count(*) from customers";
        return getValue(conn,sql);
    }

    @Override
    public Date getMaxBirth(Connection conn) throws SQLException {
```

```

        String sql = "select max(birth) from customers";
        return getValue(conn,sql);
    }
}

```

【CustomerDAOImplTest.java】

```

package com.atguigu.jdbc.dao;

import com.atguigu.jdbc.util.JDBCUtils;
import org.junit.Test;

import java.sql.Connection;
import java.util.List;

public class CustomerDAOImplTest {

    @Test
    public void test1() throws Exception {
        CustomerDAOImpl impl = new CustomerDAOImpl();
        Connection connection = JDBCUtils.getConnection();
        int deleteCount = impl.deleteById(connection, 4);
        System.out.println(deleteCount);
        JDBCUtils.close(connection);
    }

    @Test
    public void test2() throws Exception {
        CustomerDAOImpl impl = new CustomerDAOImpl();
        Connection connection = JDBCUtils.getConnection();
        List<Customer> list = impl.getAll(connection);
        for (Customer c : list) {
            System.out.println(c);
        }
        JDBCUtils.close(connection);
    }

    @Test
    public void test3() throws Exception {
        CustomerDAOImpl impl = new CustomerDAOImpl();
        Connection connection = JDBCUtils.getConnection();
        long count = impl.getCount(connection);
        System.out.println("表中共有" + count + "条记录");
        JDBCUtils.close(connection);
    }
}

```

第7章：数据库事务

7.1 数据库事务介绍

- **事务**：一组逻辑操作单元,使数据从一种状态变换到另一种状态。
- **事务处理（事务操作）**：保证所有事务都作为一个工作单元来执行，即使出现了故障，都不能改变这种执行方式。当在一个事务中执行多个操作时，要么所有的事务都被提交(commit)，那么这些修改就永久地保存下来；要么数据库管理系统将放弃所作的所有修改，整个事务回滚(rollback)到最初状态。
- 为确保数据库中数据的一致性，数据的操纵应当是离散的成组的逻辑单元：当它全部完成时，数据的一致性可以保持，而当这个单元中的一部分操作失败，整个事务应全部视为错误，所有从起始点以后的操作应全部回退到开始状态。

7.2 JDBC事务处理

- 数据一旦提交，就不可回滚。
- 数据什么时候意味着提交？
 - 当一个连接对象被创建时，默认情况下是自动提交事务：每次执行一个 SQL 语句时，如果执行成功，就会向数据库自动提交，而不能回滚。
 - 关闭数据库连接，数据就会自动的提交。如果多个操作，每个操作使用的是自己单独的连接，则无法保证事务。即同一个事务的多个操作必须在同一个连接下。
- JDBC程序中为了让多个 SQL 语句作为一个事务执行：
 - 调用 Connection 对象的 setAutoCommit(false); 以取消自动提交事务
 - 在所有的 SQL 语句都成功执行后，调用 commit(); 方法提交事务
 - 在出现异常时，调用 rollback(); 方法回滚事务

若此时 Connection 没有被关闭，还可能被重复使用，则需要恢复其自动提交状态 setAutoCommit(true)。尤其是在使用数据库连接池技术时，执行close()方法前，建议恢复自动提交状态。

【案例：用户AA向用户BB转账100】

```
public void testJDBCTransaction() {
    Connection conn = null;
    try {
        // 1.获取数据库连接
        conn = JDBCUtils.getConnection();
        // 2.开启事务
        conn.setAutoCommit(false);
        // 3.进行数据库操作
        String sql1 = "update user_table set balance = balance - 100 where user = ?";
        update(conn, sql1, "AA");

        // 模拟网络异常
        //System.out.println(10 / 0);

        String sql2 = "update user_table set balance = balance + 100 where user = ?";
        update(conn, sql2, "BB");
    } catch (Exception e) {
        rollback(conn);
    } finally {
        close(conn);
    }
}
```

```

        // 4.若没有异常，则提交事务
        conn.commit();
    } catch (Exception e) {
        e.printStackTrace();
        // 5.若有异常，则回滚事务
        try {
            conn.rollback();
        } catch (SQLException e1) {
            e1.printStackTrace();
        }
    } finally {
        try {
            //6.恢复每次DML操作的自动提交功能
            conn.setAutoCommit(true);
        } catch (SQLException e) {
            e.printStackTrace();
        }
        //7.关闭连接
        JDBCUtils.closeResource(conn, null, null);
    }
}

```

其中，对数据库操作的方法为：

```

//使用事务以后的通用的增删改操作（version 2.0）
public void update(Connection conn ,String sql, Object... args) {
    PreparedStatement ps = null;
    try {
        // 1.获取PreparedStatement的实例（或：预编译sql语句）
        ps = conn.prepareStatement(sql);
        // 2.填充占位符
        for (int i = 0; i < args.length; i++) {
            ps.setObject(i + 1, args[i]);
        }
        // 3.执行sql语句
        ps.execute();
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        // 4.关闭资源
        JDBCUtils.closeResource(null, ps);
    }
}
}

```

7.3 事务的ACID属性

1. 原子性 (Atomicity)

原子性是指事务是一个不可分割的工作单位，事务中的操作要么都发生，要么都不发生。

2. 一致性 (Consistency)

事务必须使数据库从一个一致性状态变换到另外一个一致性状态。

3. 隔离性 (Isolation)

事务的隔离性是指一个事务的执行不能被其他事务干扰，即一个事务内部的操作及使用的数据对并发的其他事务是隔离的，并发执行的各个事务之间不能互相干扰。

4. 持久性 (Durability)

持久性是指一个事务一旦被提交，它对数据库中数据的改变就是永久性的，接下来的其他操作和数据库故障不应该对其有任何影响。

7.3.1 数据库的并发问题

- 对于同时运行的多个事务, 当这些事务访问数据库中相同的数据时, 如果没有采取必要的隔离机制, 就会导致各种并发问题:
 - 脏读:** 对于两个事务 T1, T2, T1 读取了已经被 T2 更新但还没有被提交的字段。之后, 若 T2 回滚, T1 读取的内容就是临时且无效的。
 - 不可重复读:** 对于两个事务 T1, T2, T1 读取了一个字段, 然后 T2 更新了该字段。之后, T1 再次读取同一个字段, 值就不同了。
 - 幻读:** 对于两个事务 T1, T2, T1 从一个表中读取了一个字段, 然后 T2 在该表中插入了一些新的行。之后, 如果 T1 再次读取同一个表, 就会多出几行。
- 数据库事务的隔离性:** 数据库系统必须具有隔离并发运行各个事务的能力, 使它们不会相互影响, 避免各种并发问题。
- 说明: 不可重复读的和幻读很容易混淆, 不可重复读侧重于修改, 幻读侧重于新增或删除。解决不可重复读的问题只需锁住满足条件的行, 解决幻读需要锁表
- 一个事务与其他事务隔离的程度称为隔离级别。数据库规定了多种事务隔离级别, 不同隔离级别对应不同的干扰程度, **隔离级别越高, 数据一致性就越好, 但并发性越弱。**

7.3.2 四种隔离级别

- 数据库提供的4种事务隔离级别:

隔离级别	描述
READ UNCOMMITTED (读未提交数据)	允许事务读取未被其他事物提交的变更. 脏读, 不可重复读和幻读的问题都会出现
READ COMMITTED (读已提交数据)	只允许事务读取已经被其它事务提交的变更. 可以避免脏读, 但不可重复读和幻读问题仍然可能出现
REPEATABLE READ (可重复读)	确保事务可以多次从一个字段中读取相同的值. 在这个事务持续期间, 禁止其他事物对这个字段进行更新. 可以避免脏读和不可重复读, 但幻读的问题仍然存在.
SERIALIZABLE(串行化)	确保事务可以从一个表中读取相同的行. 在这个事务持续期间, 禁止其他事务对该表执行插入, 更新和删除操作. 所有并发问题都可以避免, 但性能十分低下.

- 表格从上至下, 一致性越来越好, 但是并发性越来越差。
- Oracle 支持的 2 种事务隔离级别: **READ COMMITTED**, **SERIALIZABLE**。Oracle 默认的事务隔离级别为: **READ COMMITTED**。
- Mysql 支持 4 种事务隔离级别。Mysql 默认的事务隔离级别为: **REPEATABLE READ**。

7.3.3 在MySQL中设置隔离级别

- 每启动一个 mysql 程序, 就会获得一个单独的数据库连接. 每个数据库连接都有一个全局变量 @@tx_isolation, 表示当前的事务隔离级别。
- 查看当前的隔离级别:

```
SELECT @@tx_isolation;
```

- 设置当前 MySQL 连接的隔离级别:

```
set transaction isolation level read committed;
```

- 设置数据库系统的全局的隔离级别:

```
set global transaction isolation level read committed;
```

- 补充操作:

- 创建mysql数据库用户:

```
create user tom identified by 'abc123';
```

- 授予权限

```
#授予通过网络方式登录的tom用户，对所有库所有表的全部权限，密码设为abc123.
grant all privileges on *.* to tom@'%' identified by 'abc123';

#给tom用户使用本地命令行方式，授予atguigudb这个库下的所有表的插删改查的权限。
grant select,insert,delete,update on atguigudb.* to tom@localhost
identified by 'abc123';
```

8. 章节练习与总结

8.1 章节练习

练习：创立数据库表 examstudent，表结构如下：

字段名	说明	类型
FlowID	流水号	int(10)
Type	四级 / 六级	int(5)
IDCard	身份证号码	varchar(18)
ExamCard	准考证号码	varchar(15)
StudentName	学生姓名	varchar(20)
Location	区域	varchar(20)
Grade	成绩	int(10)

向数据表中添加如下数据：

1	4	412824195263214584	200523164754000	张锋	郑州	85
2	4	222224195263214584	200523164754001	孙朋	大连	56
3	6	342824195263214584	200523164754002	刘明	沈阳	72
4	6	100824195263214584	200523164754003	赵虎	哈尔滨	95
5	4	454524195263214584	200523164754004	杨丽	北京	64
6	4	854524195263214584	200523164754005	王小红	太原	60

代码实现1：插入一个新的student 信息

请输入考生的详细信息

Type:

IDCard:

ExamCard:

StudentName:

Location:

Grade:

信息录入成功!

代码实现2：在 IDEA中建立 java 程序：输入身份证号或准考证号可以查询到学生的基本信息。结果如下：

请选择您要输入的类型：

a:准考证号

b:身份证号

c

您的输入有误！请重新进入程序。

请选择您要输入的类型：

a:准考证号

b:身份证号

a

请输入准考证号：

200523164754004

=====查询结果=====

流水号： 5

四级/六级：4

身份证号： 454524195263214584

准考证号： 200523164754004

学生姓名： 杨丽

区域： 北京

成绩： 64

请选择您要输入的类型：

a:准考证号

b:身份证号

a

请输入准考证号：

20052316475

查无此人！请重新进入程序

代码实现3：完成学生信息的删除功能

请输入学生的考号：

45135324543632

查无此人，请重新输入！

请输入学生的考号：

6342634

删除成功！

8.2 JDBC总结

总结

@Test

```
public void testUpdateWithTx() {

    Connection conn = null;
    try {
        //1. 获取连接的操作 (
        //① 手写的连接: JDBCUtils.getConnection();
        //② 使用数据库连接池: C3P0;DBCP;Druid
        //2. 对数据表进行一系列CRUD操作
        //① 使用PreparedStatement实现通用的增删改、查询操作 (version 1.0 \ version
2.0)
        //version2.0的增删改public void update(Connection conn,String sql,Object ... args)
        {}
        //version2.0的查询 public <T> T getInstance(Connection conn,Class<T> clazz,String
sql,Object ... args){}
        //② 使用dbutils提供的jar包中提供的QueryRunner类

        //提交数据
        conn.commit();

    } catch (Exception e) {
        e.printStackTrace();

        try {
            //回滚数据
            conn.rollback();
        } catch (SQLException e1) {
            e1.printStackTrace();
        }
    }

    }finally{
        //3. 关闭连接等操作
        //① JDBCUtils.closeResource();
        //② 使用dbutils提供的jar包中提供的Dbutils类提供了关闭的相关操作

    }
}
```